TRISHA ACADEMY

DevOps Career Track · Student Notes





Kubernetes Class 4

Services and Networking

Student notes - concepts, diagrams, live lab and 20 interview questions

60-90 minutes **Everyday examples** **Diagrams** **Live lab** **20 interview Q&A**

For students of TRISHA ACADEMY · Keep these notes for revision and interviews

Contents

Class 4: Services and cluster networking 3

4.1 Why we need Services 3

4.2 Ports explained 4

4.3 Service types 4

4.4 Cluster DNS 5

4.5 How pod networking works 5

4.6 Live lab: Services in action 6

Class 4 - Top 20 interview questions 7

K8S · CLASS 04 · 60-90 minutes

Services and cluster networking

By the end you can expose apps inside and outside the cluster, explain every Service type, and describe how DNS, kube-proxy and CNI make pod networking work.

What you will learn today

- Why pod IPs cannot be trusted
- Services, selectors and Endpoints/EndpointSlices
- Service types: ClusterIP, NodePort, LoadBalancer, ExternalName, headless
- Ports: port, targetPort, nodePort
- Cluster DNS with CoreDNS
- The Kubernetes networking model, kube-proxy and CNI plugins
- Lab: load balancing across pods, NodePort and LoadBalancer on minikube

Part	Topic	Time
1	Story and why Services exist	10 min
2	Service types and ports	20 min
3	DNS, kube-proxy, CNI	15 min
4	Live lab	25 min
5	Recap and interview questions	10 min

4.1 Why we need Services

Ravi's story

The orders service called the menu service at `10.244.1.5`. After a rolling update the menu pods got new IPs and every order failed with "connection refused". Ravi learned the hard way: **pod IPs change every time a pod is recreated**. He needed one address that never changes.

A **Service** gives a group of pods a stable virtual IP (ClusterIP) and a DNS name. It finds its pods using a **label selector** and load-balances traffic across the ones that are ready.

Everyday example

A restaurant prints one phone number on every menu card. Behind it, whichever staff member is free answers the call. Staff join and leave, but the number on the card never changes. The Service is that phone number; pods are the staff.

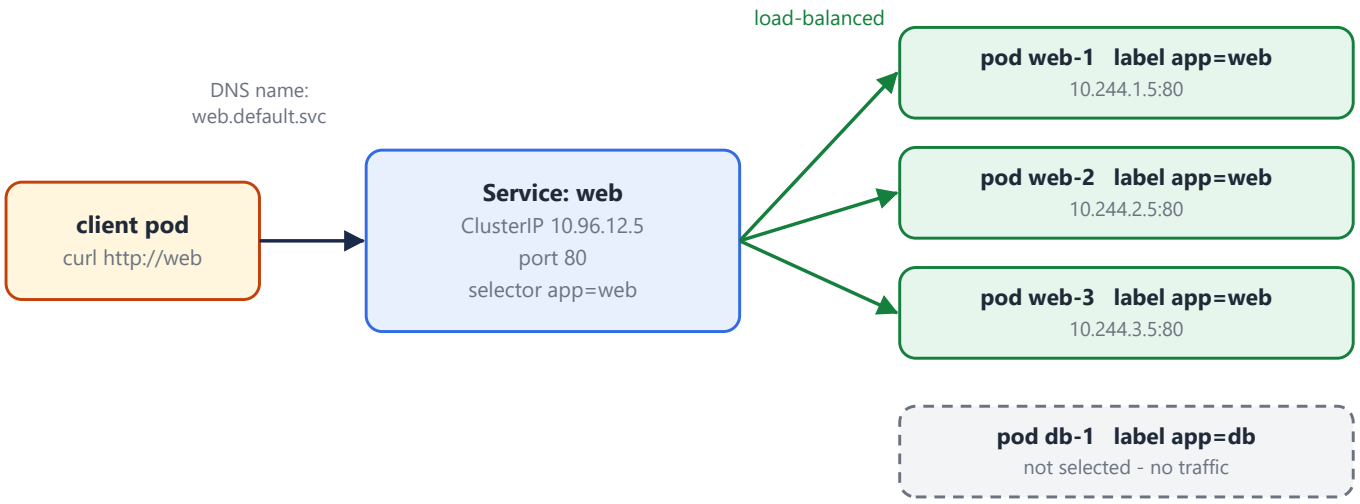


Figure 4.1 - A Service selects pods by label and spreads traffic across them

Behind the scenes Kubernetes keeps an **EndpointSlice** object listing the IP:port of every ready pod that matches the selector. If a Service has no endpoints, its selector or readiness is wrong - this is one of the most common bugs.

4.2 Ports explained

Field	Where	Example
port	The port the Service listens on (what clients call)	80
targetPort	The port on the container that receives traffic	8080
nodePort	Port opened on every node (NodePort/LoadBalancer only), range 30000-32767	30080
containerPort	Documentation in the pod spec - informational only	8080

4.3 Service types

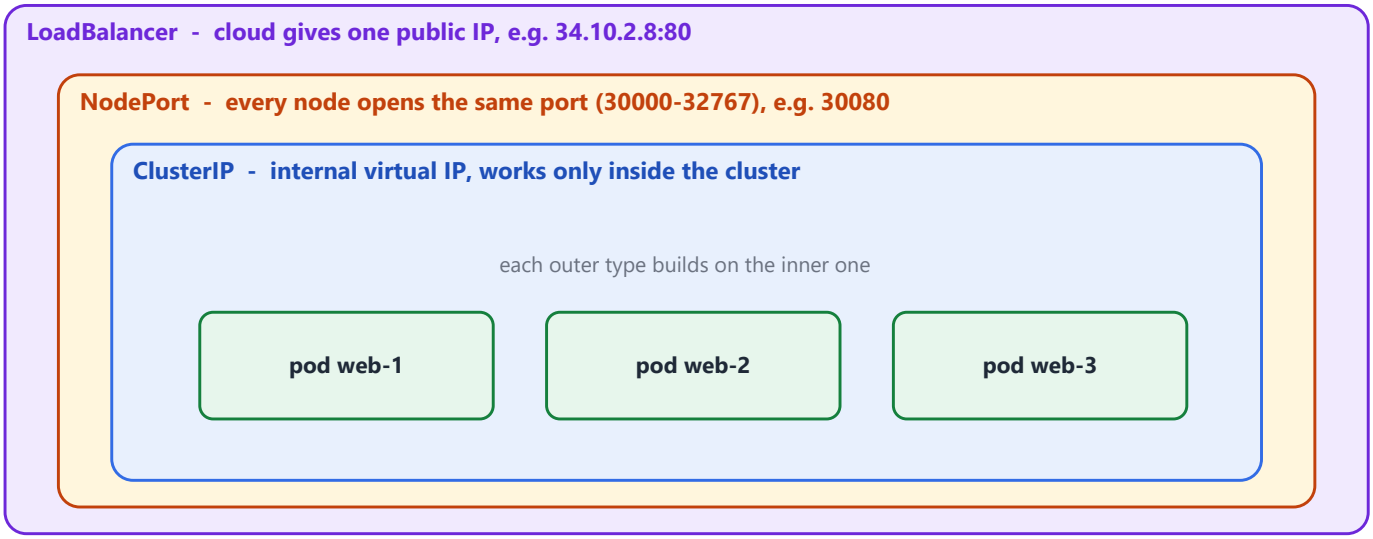


Figure 4.2 - Each Service type builds on the previous one

Type	Reachable from	Typical use
ClusterIP (default)	Inside the cluster only	Backend APIs, databases, internal microservices

Type	Reachable from	Typical use
NodePort	`<any-node-IP>:30000-32767`	Labs, on-prem setups, behind your own load balancer
LoadBalancer	Public/private IP from the cloud load balancer	Exposing an app directly on EKS/AKS/GKE
ExternalName	DNS alias (CNAME) to an outside hostname	Calling a managed DB like `db.example.com` by an internal name
Headless (`clusterIP: None`)	DNS returns pod IPs directly; no load balancing	StatefulSets, databases, clients that pick their own pod

service.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: web
spec:
  type: ClusterIP          # change to NodePort or LoadBalancer
  selector:
    app: web
  ports:
    - port: 80             # service port
      targetPort: 80       # container port
      # nodePort: 30080    # only for NodePort/LoadBalancer
```

4.4 Cluster DNS

Every cluster runs **CoreDNS**. Each Service gets a DNS record:

```
<service>.<namespace>.svc.cluster.local

web                                  # from a pod in the same namespace
web.shop                             # from another namespace
web.shop.svc.cluster.local           # fully qualified
```

Pods find CoreDNS through `/etc/resolv.conf`, which the kubelet writes into every container.

4.5 How pod networking works

Kubernetes has a simple networking model with three rules:

- Every pod gets its own unique IP address.
- Pods can reach every other pod on any node without NAT.
- Agents on a node (like the kubelet) can reach all pods on that node.

Kubernetes itself does not implement this. A **CNI plugin** (Container Network Interface) does - Calico, Flannel, Cilium, Weave, or the cloud's own (AWS VPC CNI, Azure CNI). **kube-proxy** then implements Services by writing iptables/IPVS/nftables rules on every node that translate the Service IP into a real pod IP.

> **Everyday example**
> Think of an office campus. The CNI plugin builds the roads and gives every cabin a door number. kube-proxy is the receptionist with a rule book: "calls for Accounts go to cabin 12, 14 or 19 - pick any free one".

4.6 Live lab: Services in action

We use `traefik/whoami`, a tiny web app that replies with its own pod name - perfect for seeing load balancing.

PowerShell - ClusterIP

```
kubectl create deployment whoami --image=traefik/whoami --replicas=3
kubectl expose deployment whoami --port=80 --target-port=80
kubectl get svc whoami
kubectl get endpointslices -l kubernetes.io/service-name=whoami

# call it from inside the cluster several times
kubectl run client --rm -it --image=busybox:1.36 -- sh
  wget -qO- http://whoami | grep Hostname      # repeat 5 times - names change
  nslookup whoami
  exit
```

PowerShell - NodePort and LoadBalancer

```
kubectl expose deployment whoami --name=whoami-np --type=NodePort --port=80
kubectl get svc whoami-np               # note the 3xxxx port
minikube service whoami-np --url        # gives a localhost URL on Windows; keep it open
curl.exe <url-from-previous-command>

kubectl expose deployment whoami --name=whoami-lb --type=LoadBalancer --port=80
kubectl get svc whoami-lb               # EXTERNAL-IP is <pending>
minikube tunnel                         # in a separate admin window; leave running
kubectl get svc whoami-lb               # EXTERNAL-IP now 127.0.0.1
curl.exe http://127.0.0.1
```

Now break it: change the selector so it matches nothing and see the endpoints disappear.

PowerShell

```
kubectl set selector svc whoami app=wrong
kubectl get endpointslices -l kubernetes.io/service-name=whoami   # no endpoints
kubectl delete deployment whoami; kubectl delete svc whoami whoami-np whoami-lb
```

Try it yourself

- Scale whoami to 1 and back to 5 while looping requests - the Service adapts instantly.
- Run `kubectl get svc -A` to find the `kubernetes` Service, which points at the API server, and `kube-dns` in kube-system.

Common mistake

- Confusing `port` and `targetPort`; if the app listens on 8080, targetPort must be 8080.
- Labels typo in the selector: the Service is created fine but has zero endpoints.
- Expecting to ping a ClusterIP - it is virtual and only answers on its defined ports.

Class 4 recap

- Services give pods a stable IP + DNS name and load-balance.
- ClusterIP < NodePort < LoadBalancer; ExternalName = DNS alias; headless = no IP.
- DNS: service.namespace.svc.cluster.local (CoreDNS).
- CNI gives pods IPs; kube-proxy routes Service traffic.

Class 4 - Top 20 interview questions

Try answering each question aloud before you read the model answer. Interviewers look for a clear definition, one real example, and one trade-off.

Q1. What is a Kubernetes Service and why is it needed?

An abstraction that gives a set of pods a stable virtual IP and DNS name and load-balances traffic to them. It is needed because pod IPs change whenever pods are recreated or rescheduled.

Q2. How does a Service know which pods to send traffic to?

Through its label selector. The EndpointSlice controller keeps a list of ready pods matching the selector, and kube-proxy routes traffic to those IPs.

Q3. What are the Service types?

ClusterIP (internal, default), NodePort (opens a port 30000-32767 on every node), LoadBalancer (provisions a cloud load balancer), ExternalName (DNS CNAME to an external name). A headless Service is a ClusterIP Service with `clusterIP: None`.

Q4. Difference between port, targetPort and nodePort?

port is what the Service listens on, targetPort is the container port traffic is forwarded to, and nodePort is the port opened on every node for NodePort/LoadBalancer Services.

Q5. What is a headless Service?

A Service with `clusterIP: None`. There is no virtual IP or load balancing; DNS returns the pod IPs directly. StatefulSets use it to give each pod a stable DNS name.

Q6. What is an ExternalName Service?

A Service that maps an internal name to an external DNS name via a CNAME record, with no selector or proxying - for example pointing `db` to a managed database hostname.

Q7. What are Endpoints and EndpointSlices?

Objects listing the IP:port pairs of the pods backing a Service. EndpointSlices are the newer, scalable version that splits large lists into chunks. `kubectl get endpointslices` helps debug Services.

Q8. A Service is not reachable. How do you troubleshoot?

Check the Service has endpoints, compare selector with pod labels, confirm pods are Ready, verify port/targetPort, test from a pod with wget/curl by IP and by DNS name, check CoreDNS, and check any NetworkPolicies.

Q9. How does DNS work inside Kubernetes?

CoreDNS runs in kube-system and is exposed as the kube-dns Service. The kubelet configures every pod's /etc/resolv.conf to use it. Services resolve as `<svc>.<ns>.svc.cluster.local`.

Q10. How can a pod reach a Service in another namespace?

By using the namespace in the name, e.g. `http://orders.shop` or the full `orders.shop.svc.cluster.local`.

Q11. What is kube-proxy and which modes does it support?

A node component that implements Services by programming iptables, IPVS or nftables rules that translate Service IPs to pod IPs. Some CNIs like Cilium can replace kube-proxy with eBPF.

Q12. What is CNI? Name some plugins.

Container Network Interface - a standard for plugins that give pods IPs and connect them across nodes. Examples: Calico, Flannel, Cilium, Weave, AWS VPC CNI, Azure CNI.

Q13. Explain the Kubernetes networking model.

Every pod gets a unique IP; all pods can communicate with all other pods across nodes without NAT; node agents can reach pods on their node. The CNI plugin implements it.

Q14. How do you expose an application outside the cluster?

With a NodePort or LoadBalancer Service, or an Ingress/Gateway in front of ClusterIP Services. `kubectl port-forward` works for temporary local testing.

Q15. What is the difference between NodePort and LoadBalancer?

NodePort opens a fixed port on each node; clients must know a node IP. LoadBalancer additionally asks the cloud for an external load balancer with one stable IP that forwards to the node ports.

Q16. How does a LoadBalancer Service work on bare metal or minikube?

There is no cloud controller, so EXTERNAL-IP stays pending unless you add something like MetalLB, or on minikube run `minikube tunnel`.

Q17. What is session affinity in a Service?

Setting `sessionAffinity: ClientIP` sends requests from the same client IP to the same pod. Default is None.

Q18. Can a Service exist without a selector?

Yes. You then create Endpoints/EndpointSlices manually to point to IPs outside the cluster, such as a legacy database.

Q19. Why can't you ping a ClusterIP?

A ClusterIP is a virtual IP implemented by kube-proxy rules for specific ports and protocols. No interface owns it, so ICMP ping usually does not work; test with curl on the service port.

Q20. What is the `kubernetes` Service in the default namespace?

A built-in ClusterIP Service pointing to the API server, so pods can reach the API at `https://kubernetes.default.svc`.